**UNIVERSIDAD TECNOLÓGICA METROPOLITANA**

**DESARROLLO Y GESTIÓN DE SOFTWARE**

**OPTATIVA**

**QUINTAL WARNER MICHELLE ALESSANDRA**

**Tarea 1**

**INTEGRANTES:**

Buenfil Yunes Julián Nahim

López Uicab Rossana Sofía

Martinez Rivero Allan Alexis

Sima Moo Jafet de Jesús

10° Cuatrimestre

2° Parcial

10-B

27 de octubre de 2026

REPO: https://github.com/Jess4lxy/StockAlert-project.git

## 1. Inventario de información

StockAlert almacena información sobre los usuarios, los productos, los movimientos del inventario y las alertas generadas por los niveles de existencias. Estos datos permiten mantener actualizado el inventario, identificar productos con poca disponibilidad y consultar el historial de operaciones.

| Tipo de dato | Campos | Origen | Frecuencia de actualización | Relaciones |
|---|---|---|---|---|
| Usuarios | id_usuario, nombre, correo, contrasena_hash, rol, fecha_registro, estado | Formulario de registro o alta por un administrador | Baja: cambia al modificar datos, permisos o estado de la cuenta | Un usuario registra muchos movimientos |
| Productos | id_producto, nombre, descripcion, categoria, stock_actual, stock_minimo, precio, fecha_actualizacion, estado | Formulario de administración de productos | Media/alta: stock_actual cambia con cada entrada o salida | Un producto tiene muchos movimientos y muchas alertas |
| Movimientos de inventario | id_movimiento, id_producto, id_usuario, tipo_movimiento (entrada, salida, ajuste), cantidad, fecha_movimiento, observaciones | Operaciones registradas desde la interfaz | Alta: cada operación agrega un registro; no se editan | Cada movimiento pertenece a un producto y a un usuario |
| Alertas de inventario | id_alerta, id_producto, tipo_alerta, stock_detectado, fecha_generacion, estado, fecha_atencion | Reglas del backend al detectar existencias en o bajo el mínimo | Media: una por cada evento de stock bajo | Cada alerta corresponde a un producto |

En términos de volumen, velocidad y variedad: los movimientos concentran el crecimiento y las escrituras (una por cada operación), mientras que la variedad es baja porque casi toda la información es estructurada y relacionada.

## 2. Escenarios de crecimiento

Se suponen unos 40 movimientos diarios por negocio. Las cantidades son escenarios de prueba, no cifras actuales del proyecto; los tamaños son estimaciones y los tiempos reales se medirán en la Tarea 3.

| Registros | Equivale a | Comportamiento esperado | Posibles dificultades |
|---|---|---|---|
| 1,000 | ~25 días de un negocio | Consultas en milisegundos aun sin índices; administración sencilla. | Ninguna visible: los defectos de diseño quedan ocultos. |
| 10,000 | ~8 meses de un | La base responde bien, pero | Se necesita paginación del lado |

| Registros | Equivale a | Comportamiento esperado | Posibles dificultades |
|---|---|---|---|
| | negocio | enviar todos los registros al navegador en una sola petición vuelve lenta la pantalla. | del servidor e índices en campos de búsqueda (id_producto, nombre, fecha_movimiento). |
| 100,000 | 100 negocios durante un mes | Para PostgreSQL es un volumen moderado (~15 MB estimados); el costo proviene de recorrer tablas completas en cada consulta. | Búsquedas sin índice, páginas lejanas (OFFSET) y reportes recalculados se multiplican con usuarios simultáneos. |

### *2.1. Consultas que podrían volverse lentas*

#### Consulta 1. Búsqueda de movimientos por nombre de producto.

```
SELECT m.id_movimiento, m.fecha_movimiento, m.tipo_movimiento, m.cantidad, p.nombre
FROM movimientos m JOIN productos p ON p.id_producto = m.id_producto
WHERE p.nombre ILIKE '%' || $1 || '%'
ORDER BY m.fecha_movimiento DESC
LIMIT 20 OFFSET $2;
```

El comodín inicial impide usar un índice B-tree, por lo que se revisan todos los productos; PostgreSQL no crea índices automáticos sobre claves foráneas, así que la unión recorre toda la tabla de movimientos, y para devolver 20 filas debe ordenar todas las coincidencias. Se ejecuta con cada búsqueda del usuario.

**Propuesta de optimización:** crear un índice de trigramas (pg_trgm) sobre productos.nombre y un índice compuesto sobre (id_producto, fecha_movimiento).

#### Consulta 2. Historial de movimientos de un producto durante un periodo.

```
SELECT id_movimiento, id_producto, id_usuario, tipo_movimiento, cantidad, fecha_movimiento
FROM movimientos
WHERE id_producto = $1 AND fecha_movimiento BETWEEN $2 AND $3
ORDER BY fecha_movimiento DESC;
```

Con el tiempo la tabla de movimientos acumula numerosos registros; sin índices apropiados, el sistema revisa una gran cantidad de filas para localizar las operaciones solicitadas.

**Propuesta de optimización:** índice compuesto sobre (id_producto, fecha_movimiento), paginación de resultados y límite al periodo solicitado.

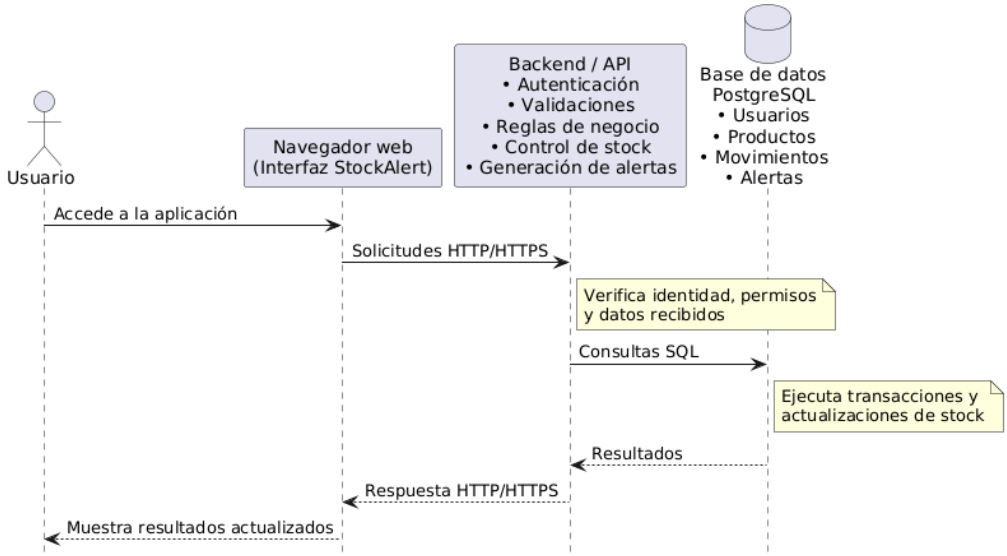
## 3. Comparación tecnológica: PostgreSQL vs. MongoDB

| Criterio | PostgreSQL | MongoDB |
|---|---|---|
| Estructura | Relacional: tablas, filas y columnas con esquema definido. | No relacional: documentos similares a JSON dentro de colecciones. |
| Relaciones | Claves primarias y foráneas entre tablas. | Referencias o documentos anidados; requieren un diseño específico. |
| Consultas | SQL con filtros, uniones (JOIN) y agregaciones. | Lenguaje propio y pipeline de agregación; las uniones ($lookup) son más costosas. |
| Integridad | Restricciones y transacciones ACID para mantener la consistencia. | Validación de esquemas y transacciones multidocumento, con consideraciones de diseño. |
| Escalabilidad | Vertical, réplicas de lectura y particionamiento; distribución horizontal con extensiones (p. ej., Citus). | Vertical, replicación y fragmentación horizontal nativa (sharding). |
| Costos | Código abierto; el costo depende de la infraestructura y los servicios contratados. | Edición comunitaria y servicios comerciales (Atlas); el costo depende del servicio elegido. |
| Integración | Controladores para muchos lenguajes; en Node.js: pg, Prisma o Drizzle. | Controladores para muchos lenguajes; en Node.js: driver oficial o Mongoose. |
| Uso recomendado en StockAlert | Inventario, movimientos, usuarios y alertas con relaciones consistentes. | Datos con estructura variable que justifiquen un modelo documental. |

Para StockAlert, PostgreSQL facilita relacionar productos, usuarios, movimientos y alertas, actualizar el inventario mediante transacciones y optimizar consultas con índices; su desventaja es que el esquema debe diseñarse antes y modificarse de forma planificada. MongoDB ofrece estructuras flexibles y distribución horizontal, pero las relaciones requieren un modelado específico, la duplicación de datos puede generar inconsistencias y las operaciones sobre varios documentos necesitan transacciones adicionales.

## 4. Diseño de la solución

### *4.1. Diagrama del flujo de información*




1. El usuario accede a StockAlert desde el navegador (interfaz en Vue 3).
2. La interfaz envía solicitudes HTTP/HTTPS a la API (Node.js + Express) para consultar productos, registrar movimientos o recuperar alertas; los listados se solicitan por páginas.
3. El backend verifica la identidad del usuario, sus permisos y los datos recibidos.
4. Si la solicitud es válida, ejecuta consultas SQL parametrizadas y transacciones en PostgreSQL (en desarrollo, un contenedor Docker local).
5. La base de datos devuelve los resultados y el backend responde al navegador con la información actualizada.

### *4.2. Modelo de datos*

El modelo está compuesto por cuatro entidades: Usuarios, Productos, Movimientos y Alertas.

| Entidad | Campos y tipos | Llaves y restricciones |
|---|---|---|
| Usuarios | id_usuario INTEGER, nombre VARCHAR(100), correo VARCHAR(150), contrasena_hash TEXT, rol VARCHAR(30), fecha_registro TIMESTAMPTZ, estado BOOLEAN | PK id_usuario; correo único; rol: admin o almacen |
| Productos | id_producto INTEGER, nombre VARCHAR(150), descripcion TEXT, categoria VARCHAR(80), stock_actual INTEGER, stock_minimo INTEGER, precio NUMERIC(12,2), fecha_actualizacion TIMESTAMPTZ, estado BOOLEAN | PK id_producto; stock_actual, stock_minimo y precio no negativos |
| Movimientos | id_movimiento BIGINT, id_producto INTEGER, id_usuario INTEGER, tipo_movimiento VARCHAR(20), cantidad INTEGER, fecha_movimiento TIMESTAMPTZ, observaciones | PK id_movimiento; FK a productos y usuarios; tipo: entrada, salida o ajuste; cantidad mayor que cero |

| Entidad | Campos y tipos | Llaves y restricciones |
|---|---|---|
| | TEXT | (distinta de cero en ajustes) |
| Alertas | id_alerta BIGINT, id_producto INTEGER, tipo_alerta VARCHAR(30), stock_detectado INTEGER, fecha_generacion TIMESTAMPTZ, estado VARCHAR(20), fecha_atencion TIMESTAMPTZ | PK id_alerta; FK a productos; estado: pendiente, atendida o cerrada |

Un usuario registra muchos movimientos; un producto aparece en muchos movimientos y puede generar múltiples alertas. Las relaciones se implementan con claves foráneas y, para conservar el historial, un producto se desactiva (estado = falso) en lugar de eliminarse físicamente. El script del modelo está en db/schema.sql del repositorio.

## 5. Decisión del equipo

El equipo selecciona PostgreSQL como sistema de gestión de bases de datos para StockAlert, debido a que el proyecto necesita almacenar información estructurada sobre usuarios, productos, movimientos de inventario y alertas. Esta tecnología permite establecer relaciones mediante claves primarias y foráneas, realizar consultas SQL y utilizar transacciones para que el registro de un movimiento y la actualización del stock ocurran juntos.

En cuanto al volumen, PostgreSQL puede administrar desde miles hasta millones de registros, siempre que se configure adecuadamente y se optimicen las consultas. Respecto a la velocidad, permite incorporar índices, paginación y análisis de ejecución para mejorar los tiempos de respuesta. Además, es de código abierto, se ejecuta localmente con Docker y se integra con Node.js. Sin embargo, uno de los riesgos es la disminución del rendimiento conforme crece la tabla de movimientos o se ejecutan consultas ineficientes.

Para mitigarlo, el equipo implementará índices apropiados, paginación del lado del servidor, mediciones de rendimiento y copias de seguridad periódicas; si la demanda crece considerablemente, se evaluará particionar los movimientos por fecha y ampliar los recursos del servidor.

## Conclusión

El análisis de crecimiento demuestra que la cantidad de registros no es el único factor que determina el rendimiento: también influyen la estructura de las tablas, la eficiencia de las consultas, los índices y la forma de entregar los datos al navegador. La comparación tecnológica justifica la selección de PostgreSQL por su capacidad para administrar datos relacionados y mantener la consistencia de las operaciones de inventario.

Respecto a la pregunta guía, una base de datos adecuada para un prototipo puede no serlo para miles de usuarios porque, con pocos registros, las consultas sin índices ni paginación responden rápido y ocultan sus defectos; con más datos y usuarios simultáneos, esos atajos se convierten en recorridos completos de tablas y respuestas lentas. Por ello, StockAlert parte de un modelo relacional bien definido y de una estrategia de crecimiento gradual que se validará con mediciones en las siguientes tareas.